MAVE: A Product Dataset for Multi-source Attribute Value Extraction

Li Yang*,[1], Qifan Wang*,[1], Zac Yu[2], Anand Kulkarni[2], Sumit Sanghai[1], Bin Shu[2], Jon Elsas[2], Bhargav Kanagal[1]
[1]Google Research, Mountain View, USA [2]Google, Pittsburgh, USA
{lyliyang,wqfcr,zacyu,anandkulkarni,sumitsanghai,bins,jelsas,bhargav}@google.com

ABSTRACT

Attribute value extraction refers to the task of identifying values of an attribute of interest from product information. Product attribute values are essential in many e-commerce scenarios, such as customer service robots, product ranking, retrieval and recommendations. While in the real world, the attribute values of a product are usually incomplete and vary over time, which greatly hinders the practical applications. In this paper, we introduce MAVE, a new dataset to better facilitate research on product attribute value extraction. MAVE is composed of a curated set of 2.2 million products from Amazon pages, with 3 million attribute-value annotations across 1257 unique categories. MAVE has four main and unique advantages: First, MAVE is the largest product attribute value extraction dataset by the number of attribute-value examples. Second, MAVE includes multi-source representations from the product, which captures the full product information with high attribute coverage. Third, MAVE represents a more diverse set of attributes and values relative to what previous datasets cover. Lastly, MAVE provides a very challenging zero-shot test set, as we empirically illustrate in the experiments. We further propose a novel approach that effectively extracts the attribute value from the multi-source product information. We conduct extensive experiments with several baselines and show that MAVE is an effective dataset for attribute value extraction task. It is also a very challenging task on zero-shot attribute extraction. Data is available at *https://github.com/google-research-datasets/MAVE*.

CCS CONCEPTS

• **Computing methodologies** → **Information extraction**.

KEYWORDS

attribute value extraction, open tag extraction, zero-shot learning

ACM Reference Format:
Li Yang*,[1], Qifan Wang*,[1], Zac Yu[2], Anand Kulkarni[2], Sumit Sanghai[1], Bin Shu[2], Jon Elsas[2], Bhargav Kanagal[1]. 2022. MAVE: A Product Dataset for Multi-source Attribute Value Extraction. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining (WSDM '22), February 21–25, 2022, Tempe, AZ, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3488560.3498377

* Equal contributions.

WSDM '22, February 21–25, 2022, Tempe, AZ, USA

ACM ISBN 978-1-4503-9132-0/22/02…$15.00
https://doi.org/10.1145/3488560.3498377

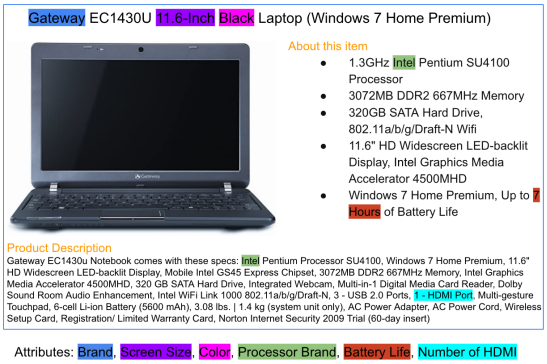

Figure 1: An example of the product profile on an e-Commerce platform. It consists of multiple information sources, including a product title, a product description and several other sources.

1 INTRODUCTION

Product attributes are critical product features which form an essential component of e-commerce platforms today. They provide useful details of the product, which help customers to make purchasing decisions and facilitate retailers on many applications, such as product search [1, 2], product recommendation [25, 50], product representation [34, 39] and question answering system [11, 23]. However, for most retailers, product attributes are often noisy and incomplete with a lot of missing values, which has been shown in many previous studies [54, 63]. Therefore, it is an important research problem to supplement the product with missing values for attributes of interest. The problem of attribute value extraction is illustrated in Figure 1. It shows the profile of a Laptop product as an example, which consists of a title, a product description and several other information sources. It also shows several attribute values that could be extracted.

There has been a lot of interest in this topic, and a plethora of research [4, 5, 10, 12, 43, 46, 61] in this area both in academia and industry. Early works on this problem are rule based approaches [6, 13, 51], which utilize domain-specific knowledge to design regular expressions. Several other approaches such as [36, 44, 58] formulate the attribute value extraction as an instance of named entity recognition (NER) problem [37], and build extraction models to identify the entities/values from the input text. With the recent

advance in natural language understanding, sequence tagging [54, 56, 59, 62] based approaches have been proposed, which achieve promising results. Attribute value extraction approaches rely on a rich dataset to help them learn to extract the attribute values from the product profile. Existing attribute datasets are created from different e-commerce platforms, such as Amazon [28, 59], AliExpress [54, 56], JD [63], etc. However, there are several major limitations:

- The scales of current public datasets are not large. In fact, many of the datasets used in previous works are not even publicly available. Moreover, existing datasets, such as the MAE one in Table 1, are very noisy and cannot be used without dense pre-processing and cleanup. Researches have shown that having a large and accurate attribute-value dataset can significantly improve model performance.
- The product profiles are relatively simple. Most product context is formed from a single source, i.e., product title or description, resulting in short sequences. In real world applications, products are usually represented by multiple sources, such as title, description, specification, table, review, image, etc, forming much longer sequences.
- The attributes and values are not diverse. The lack of the data diversity impedes research in the zero-shot/few-shot attribute value extraction, which has huge potential impact on unseen data. With the rapid expansion of e-commerce, new products with new capabilities are being released constantly which means the model needs to gracefully adapt to new attributes.

To address these challenges and advance research on product attributes, we present the Multi-source Attribute Value Extraction (MAVE) dataset. MAVE is created by first extracting the raw values from Amazon Review pages [38] on a pre-defined set of attributes. We then apply rigorous filtering and post-mapping to only retain high quality extractions. The resulting dataset contains over 2.2 million products distributed into 1257 different product categories, with 3 million attribute-value annotations - making MAVE the largest attribute value extraction dataset at the time of writing. Moreover, MAVE provides more complete product profiles from multiple information sources, including product title, descriptions, features, specifications and key-value pairs, which enhance the attribute coverage. For example, in Figure 1, the value *7 Hours* for the attribute *Battery Life* is from the product specifications. Furthermore, MAVE contains a rich and diverse set of attributes and values. In particular, there are 2535 unique attributes with 100K unique values in the MAVE dataset. It is worth pointing out that by leveraging the human rules based matching, MAVE is able to ensure a high quality bar with 97.8% precision (manual check) over 1000 random sampled attribute-value annotations.

In this paper, we also propose a novel Multi-source Attribute Value Extraction model via Question Answering (MAVEQA). MAVEQA uses the recent ETC [3] model as the backbone to effectively extract the attribute value from the multiple information sources with the long sequences. We conduct an extensive set of experiments on the MAVE dataset over several state-of-the-art baselines (including the proposed MAVEQA). The experimental results demonstrate the potency of the data. They also show that MAVE is a challenging dataset on attribute value extraction for various attributes, especially those zero-shot attributes.

Dataset	#products	#annotations	#sources	public
MAE [17]	600K	2.2M	2	Yes
OpenTag [62]	10K	13K	2	No
MEPAVE [63]	34K	87K	2	Yes
AE-110K [56]	50K	110K	1	Yes
AdaTag [59]	333K	410K	Multiple	No
MAVE	2.2M	3M	Multiple	Yes

Table 1: Existing datasets for attribute value extraction.

2 RELATED WORK

2.1 Attribute Value Extraction Models

Rule-based extraction methods [13, 37, 51] are among those early works on attribute value extraction. They use a domain-specific seed dictionary or vocabulary to identify key phrases and attributes. Ghani et al. [12] predefine a set of attributes to extract the corresponding values. Wong et al. [55] extract attribute-value pairs from the semi-structured text. Shinzato and Sekine [48] design an unsupervised method to extract attribute values from product description. Several rule-based and linguistic approaches [6, 35] leverage syntactic structure of sentences to extract dependency relations. Several NER based systems [8, 29, 36, 44] were proposed for extracting product attributes and values. However, these rule-base and domain-specific methods suffer from limited coverage and closed world assumptions.

With the development of deep neural networks, various neural network methods have been proposed and applied in sequence tagging successfully. Huang et al. [15] are the first to apply the BiLSTM-CRF model to a sequence tagging task. But it employs heavy feature engineering to extract character-level features. Lample et al. [24] utilize BiLSTM to model both word-level and character-level information rather than hand-crafted features. Chiu and Nichols [7] model character level information using convolutional neural network (CNN), which achieves competitive performance and has been extended in [32] with an end to end LSTM-CNNs-CRF model.

Recently, several approaches employ sequence tagging models for attribute value extraction. Kozareva et al. [22] adopt BiLSTM-CRF model to tag several product attributes from search queries with hand-crafted features. Furthermore, Zheng et al. [62] develop an end-to-end tagging model utilizing BiLSTM, CRF, and attention mechanism without any dictionary. More recently, Xu et al. [56] adopt only one global set of BIO tags for any attributes to scale up the model, which explicitly models the semantic representations for attribute and product title. Most recently, several approaches [31, 54, 59] formulate the problem as a question answering (QA) [16, 21] task. Wang et al. [54] propose to jointly encode both the attribute and the product context with cross-attention model, and then decode the value span with a BIO tagging. It is worth mentioning that there are also methods that work on multimodal attribute value extraction [17, 28, 63], which incorporate the visual features using CNN [30] to boost coverage of the attributes.

Relation extraction [40, 42, 47, 49, 53, 60] is also relevant to attribute value extraction. Relation extraction refers to the task of extracting relational tuples and putting them in a knowledge base. The tuple usually corresponds to a subject, a predicate and an object. Attribute value extraction can be thought of as the problem where the subject is known (the product), and given the attribute (the relation) extract the value. However, relation extraction has traditionally focused on extracting relations from sentences relying on entity linking systems to identify the subject/object and building models to learn the predicates in a sentence [5, 26]. Whereas in attribute value extraction [41, 45], usually the predicates (the attribute names) rarely occur in the product title or the description, and entity linking is very hard because the domain of all entities/values is unknown.

2.2 Attribute Value Extraction Datasets

Although product attribute value extraction is an important research problem, which has raised a lot of attention in recent years, as far as we know, there is no universal dataset for evaluating the AVE models. This is one of the motivations of this work, to establish such a benchmark dataset that could serve as a rich resource to drive research in attribute value extraction. However, there are a few datasets that are used in previous AVE approaches. MAE [17] is among the early datasets that are applied to this space. MAE is composed of mixed media data over 2 million product items, obtained by running the Diffbot Product API on over 20 million web pages from 1068 different commercial websites. The attribute-value pairs are automatically extracted from tables present on product webpages without any verification, which are very noisy - only less than 30% values can be found in about 600K products. OpenTag [62] is one of the pioneer work that models this problem as sequence tagging with LSTM. However, there are only five attributes in this data with 13K attribute value annotations. MEPAVE [57, 63] is a multimodal dataset with textual product descriptions and images. It contains 34K products collected from a JD e-commerce platform. The 87K attribute value annotations are generated by crowd sourcing annotators. AE-110K [56] is a public dataset collected from AliExpress Sports & Entertainment category. It is composed of 110K attribute value annotations obtained from Item Specific in 50K product titles. Despite the long-tailed attribute distribution (7650 of 8906 attributes occur less than 10 times), this dataset has been used in several recent works [19, 54]. AdaTag [59] builds a dataset by collecting product profiles with multiple source representations from the public web pages at Amazon.com. The 410K attribute value annotations are generated from 333K products in a similar way to the previous work [20, 56, 62]. Unfortunately, this dataset is not publicly available. There are many other works [18, 33] that use their own datasets, which are all small data with limited attributes. We summarize the existing datasets in Table 1.

3 MAVE: MULTI-SOURCE ATTRIBUTE VALUE EXTRACTION DATASET

Our MAVE dataset is derived from a public product collection - the Amazon Review Dataset [38][1] by generating and adding attribute-value annotations to the products. In this section, we describe our pipeline for creating MAVE, and also provide detailed statistics from various aspects.

Source	**Text**
title	Ty Beanie Baby Ants the Anteater
description 1	Ty Beanie Babies Ants the Anteater. Approximately 8" long, 4" tall. Birthday: November 7, 1997 Poem: Most anteaters love to eat bugs But this little fellow gives big hugs. He'd rather dine on apple pie Than eat an ant or harm a fly.
description 2	This most unusual Beanie Baby is brimming with personality. Ants was born November 7, 1997. His poem reads: Most anteaters love to eat bugs But this little fellow gives big hugs He'd rather dine on apple pie Than eat an ant or harm a fly! This long-snouted guy, balancing on his long tail, is just adorable. His head and tail are gray; his middle is three stripes white, black, and white; and he has sweet black button eyes. His tiny gray felt ears really give this guy some charm. Surface wash only.
feature 1	Ty beanie baby - Ants the anteater
feature 2	Birthday: November 7, 1997
feature 3	Approx 8" long
price	$6.06
brand	TY
Attribute	**Values**
Toy Maker	{Beanie Baby, title, span (3, 13)}, {Beanie Babies, description 1, span (3, 15)}, {Beanie Baby, description 2, span (18, 28)}, {beanie baby, feature 1, span (3, 13)}

Table 2: An example of a product profile with annotations. Each source corresponds to a field of the product.

3.1 Product Profiles

The products from the Amazon Review Dataset contain multiple information sources. For each product, we organize the 'title', 'description', 'feature', 'price', and 'brand' into a structured product profile. We performed the following cleaning of the texts to ensure a high quality of the attribute value annotations.

- Remove the html tags, e.g., script and style tags, within the product profile by using BeautifulSoup [2] as well as a few hard-coded rules.
- Remove extra whitespaces and invalid text unicode characters.
- Remove products with a total number of words less than 20.
- Remove products without a title.

Table 2 shows an example of one product profile after the preprocessing.

3.2 Product Attributes

Before describing how to generate and associate attribute values to product profiles, we first discuss the structure of the attributes.

3.2.1 Attributes are category specific. Products can be classified into different categories, e.g. *Shoes*, *Books*, *Mobile Phones*. For each category, a set of attributes are pre-defined by our human raters. For example, the *Shoes* category contains attributes like *Type*, *Style*, *Heel Height*, etc.; The *Books* category has attributes *Type*, *Format*, *Fiction Form*, etc.; The *Mobile Phones* category has attributes *Operating System*, *Battery Life*, *Screen Size*, etc. It is worth pointing out that certain attributes are more general, which are shared across multiple categories, e.g., *Type*, *Style*. But some attributes are more specific and apply to only a few categories, e.g. *Battery Life*, *Fiction Form*.

[1] https://nijianmo.github.io/amazon/index.html

[2] https://www.crummy.com/software/BeautifulSoup/bs4/doc/

Counts	Positives	Negatives
# products	2226509	1248009
# product-attribute pairs	2987151	1780428
# products with 1-2 attributes	2102927	1140561
# products with 3-5 attributes	121897	99896
# products with ≥6 attributes	1685	7552
# unique categories	1257	1114
# unique attributes	705	693
# unique category-attribute pairs	2535	2305

Table 3: Overall statistics of the MAVE dataset.

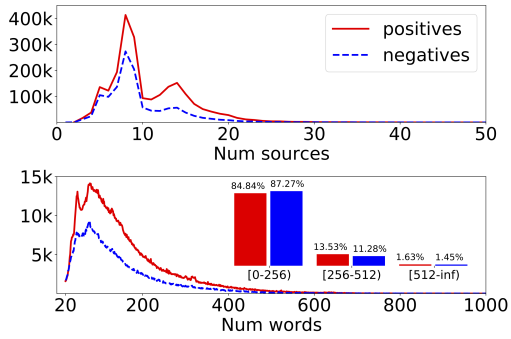

Figure 2: Distributions on # sources and # words for positive and negative sets. The inset figure is the percentage in each # words buckets within each sets.

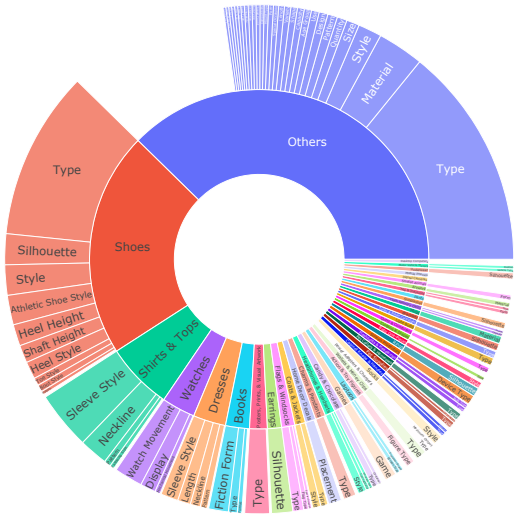

Figure 3: Number of positive example distributions on categories and attributes. Inner and outer circles are for categories and attributes, respectively. Empty blocks indicate the category-attribute contains too few examples to show.

Moreover, different attributes from different categories can share similar meanings, for example, *Resolution* for *Digital Cameras* and *Rear Camera Resolution* for *Mobile Phones*. The correlation among attributes empowers attribute value extraction models to transfer knowledge from one attribute to another.

3.2.2 Attribute values are category specific. For attributes shared across multiple categories, the definition of attribute values may also vary. For example, the *Type* for *Books* could be *Fiction* or *Non-fiction*, but the *Type* for *Shoes* could be *Sandals*, *Slippers*, etc. Note that some of the attributes have discrete or categorical values, e.g. *Style*, *Type*, which are associated with a set of well defined attribute values. On the other side, some attributes have measure, i.e., numerical values with units, e.g. *Heel Height*, *Screen Size*.

3.3 Attribute Value Annotations

In this section, we present the details of generating attribute value annotations on the product profiles. In the following discussion, we refer to an example as a tuple of (product, category, attribute).

3.3.1 Category classification. We first run a category classifier on a product to predict the category. The category vocabulary is defined by human experts, and the category classifier is a multi-class classification model trained on an existing large dataset containing category labels. We remove the products with predicted category probability lower than a threshold of 0.5 to ensure a high quality of category classifications.

3.3.2 Attribute value span extraction. As aforementioned, for each category, a set of attributes are pre-defined. For each attribute in a category, a set of extraction rules are also defined by our human raters to extract value spans in product profiles and map to normalized attribute values. We do not apply the extraction rules directly to the products because the extraction rules are not complete - they are designed for extracting specific formats of attribute values, resulting in low coverage extractions. Moreover, the extraction rules are not designed to extract all the attribute value spans in the context. Instead, we first train an ensemble of five versions of AVEQA [54] models using a large amount of silver data from the extraction rules as well as human verified gold data. These models vary in terms of random initialization, training data version, pre-post processing, etc. The input of these models is a concatenation of WordPiece tokenized category, attribute and product context. The outputs are token level attribute value spans in the product sources. We conduct inference on the cleaned Amazon product profiles using the five trained models. To ensure a high precision of the extractions, we further apply a simpler set of human extraction rules (created by human raters) to map the predicted spans to normalized attribute values, and remove the extractions that can not be mapped. The final extractions are aggregated from these five models into a positive set and a negative set as follows. For each example,

- If the normalized attribute values from all five models are identical, the example is added to the positive set. The union of the extracted spans from all five models forms the span set of the attribute values.
- If no span is extracted from either model, and there is no extracted span from extraction rules, the example with empty value spans is put into the negative set, meaning no value for this attribute.

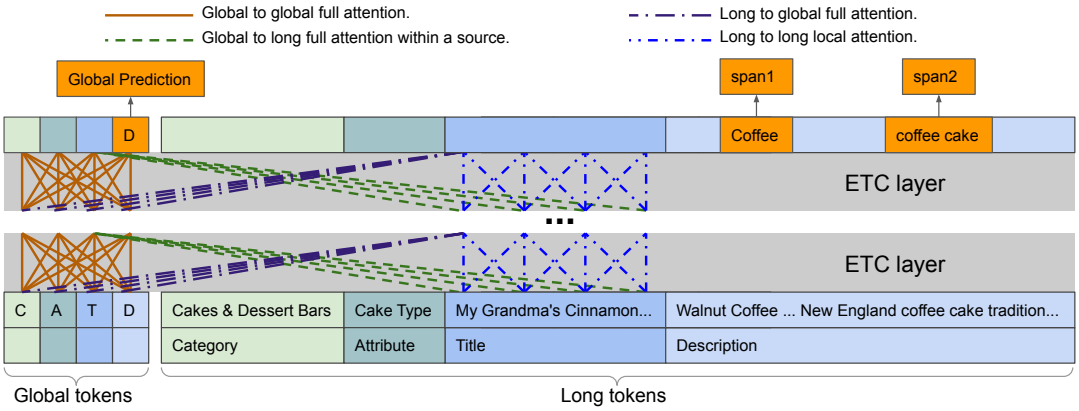

Figure 4: Overall MAVEQA model architecture.

The aggregation ensures a low false positive rate in the positive set and a low false negative rate in the negative set. However, we still generated much more negative examples compared with positive examples. Therefore, we further down sample the negative examples by restricting each category-attribute pair having at most 5K negative examples.

The spans of the attribute values are very useful information in attribute value extraction research as well as questions answering systems. In this work, we map the token level attribute value spans to character level spans in the original sources following the method used in the BERT paper for the SQuAD task [9]. If two spans overlap, we keep the one with the smallest begin index. However, in the MAVE dataset, we observe the spans are distributed sparsely with rare overlapping. The resulting (product, category, attribute, value spans) tuples are stored as an example in the MAVE dataset.

3.4 Data Statistics

The statistics of the MAVE dataset is reported in Table 3. The original Amazon Reivew dataset contains 14.7M products. After the processing and filtering as described in Section 3.1 and 3.3, we have 2.2M products with 3M product-attribute annotations in the positive set, and 1.2M products with 1.8M product-attribute annotations in the negative set. In the positive set, the majority of products have one or two attributes, but there are still 100k+ products with more than three attributes.

3.4.1 Multi-source and Sequence length statistics. A product profile contains multiple sources, the distributions on number of sources and total number of words are shown in Figure 2, where a basic white space tokenizer is used to split the texts in all sources into words. It can be seen from the figures that the total number of words in a product is much longer than the typical number of words in a product title or description, which means the dataset contains much richer information from the multi-source product profiles. We observe that there is also a small but non-negligible portion of products (36k positives, 18k negatives) with sequence length longer than 512, which can serve as a data source for benchmarking models designed for long sequences.

3.4.2 Attributes statistics. The counts of unique categories, category-attribute pairs can be found in Table 3. The average number of attributes per category is roughly 2, and the average number of categories an attribute shared across is 3. We visualize the number of positive example distributions on categories and attributes in Figure 3. It is clear that the head categories in general contain more attributes than average.

4 MULTI-SOURCE ATTRIBUTE VALUE EXTRACTION MODEL

The product profiles in MAVE dataset are composed of multiple text sources. Existing approaches might not work well on MAVE because: 1) They do not model the multi-source structure of the product, but simply treat product profile as one single text sequence by concatenating the texts from all sources. However, different source might carry different information, e.g., some attributes might appear more often in product title, while certain attributes are only mentioned in product specifications. 2) They are not able to deal with products with very long sequences. Long inputs are trimmed to fit into their models. In this work, to address these challenges, we propose a novel approach of Multi-source Attribute Value Extraction via Question Answering (MAVEQA), which effectively and efficiently models the products with structure and long profiles.

4.1 Model Overview

Following our previous work AVEQA [54], we formulate the attribute value extraction task as a question answering problem. In particular, each attribute is treated as a question, and we seek the best answer span in the product context that corresponds to the value. The overall model architecture is shown in Figure 4. Essentially, our model consists of three main components, an input layer, a contextual encoder and an output layer. In the following sub-sections, we present each component separately in detail.

4.2 Input Layer

Different from previous models [54, 56, 59], our model treats each product source as an individual sequence, and assigns a global node/token to each of them. For example, in Figure 4, the product

has two sources, i.e., one title and one description. By also regarding the attribute and category as two sources, our model assigns four global tokens to represent these four input sources respectively. The global token can be viewed as a summarization of its corresponding source. In the input layer, every word in each input source is converted into a d-dimensional embedding vector. This embedding is obtained by concatenating the word embedding and the source embedding. Each global token will also be initialized with a global token embedding. Note that all the embeddings are trainable in our approach.

4.3 Encoder

In MAVEQA, we adopt the ETC encoder [3] to generate the contextual embeddings for the global and long input tokens. ETC is an extension of the Transformer [52] encoder to better capture the structure and long input, which is a natural fit to our task. The encoder in our model is a stack of identical ETC layers. There are four attention mechanisms in each ETC layer:

- Global to global attention: each global token attends to all other global tokens, which allows information to flow among global tokens.
- Global to long attention: each global token attends to all long tokens in the corresponding source. Each global token is similar to a CLS token in the BERT model [9], which summarizes the corresponding source.
- Long to global attention: each long token in a source attends to all global tokens. This attention enables knowledge passing from other sources to the token in this source.
- Long to long attention: each long token will only attend to the long tokens within a local radius from the same source. This local attention pattern ensures the computational efficiency of the encoder.

4.4 Output Layer

In the output layer, a single dense layer with sigmoid activation is applied to the output long token embeddings, which computes a probability for every long token in all sources excluding category and attribute (as we are not seeking for values there) to predict whether the token is in an attribute value span. Similarly, the global token embeddings can be used to predict whether the corresponding source contains an attribute value or not. During training, we use sigmoid cross-entropy loss function on both long and global tokens.

5 EXPERIMENTAL RESULTS

5.1 Baseline models

We adopt the following models as baselines on the MAVE dataset.

- **OpenTag** [62] uses a BiLSTM-Attention-CRF architecture with sequence tagging strategies. OpenTag does not encode the attribute and thus builds one model per attribute. In order to scale up to an arbitrary number of attributes, similar to SUOpenTag model [56], we concatenate category, attribute, and source tokens in the input and make it attribute dependent. This attribute-dependent version of OpenTag is referred to **ADOpenTag**.

Attributes	Train		Eval	
	# positive	# negative	# positive	# negative
Selected Head Attributes				
Type	805401	86661	100763	10738
Style	138124	47899	17173	6151
Material	93701	30204	11794	3762
Size	21906	13323	2836	1690
Capacity	15088	9149	1881	1161
Selected Tail Attributes				
Black Tea Variety	124	3434	15	439
Staple Type	135	249	22	33
Web Pattern	163	1110	19	144
Cabinet Configuration	158	112	24	15
Power Consumption	132	140	16	27
All Attributes				
All Attributes	2388842	1425936	298696	176978
Zero-shot Attributes				
Special Occasion	0	0	16142	105908
Resolution	0	0	9509	5128
Compatibility	0	0	6828	136
Number of Sinks	0	0	349	446
Food Processor Capacity	0	0	261	807

Table 4: Number of positive and negative examples in train and eval sets for the selected attributes, all attributes and zero-shot attributes benchmarks.

- **AVEQA** [54] formulates the attribute value extraction as a question answering task. This model jointly encodes both attribute and product context with BERT encoder. It achieves the state-of-the-art results in the AE-110K dataset.
- **MAVEQA** is the new model proposed in this work, which adopts ETC [3] encoder to encode structure and longer input sequences.

For all these baselines, we use the same output layer which is a single dense layer with sigmoid activation to predict token scores, and we use the same English uncased WordPiece vocabulary as in BERT [9]. More implementation details are provided in the appendix.

5.2 Benchmarks

We arrange the following setups for benchmark:

- **Selected Attributes** To demonstrate the baseline model performance on individual attributes, we randomly select a set of attributes. In particular, we randomly select five attributes from the head ones that contain a large number of attribute-value annotations in the dataset. We also randomly select five tail attributes that have very few examples in the dataset.
- **All Attributes** In order to evaluate the models' capability of scaling up to an arbitrary number of attributes, we also conduct experiments of all baselines on all attributes.
- **Zero-shot Attributes** To further examine the generalization ability of the models, we randomly select five attributes and holdout these attributes from training, i.e., none of these attributes are seen during training. We refer them to zero-shot attributes as they are used for evaluating zero-shot extraction.

Attributes	(AD)OpenTag			AVEQA			MAVEQA		
	P (%)	R (%)	F (%)	P (%)	R (%)	F (%)	P (%)	R (%)	F (%)
Type	91.09	89.06	90.06	98.51	98.58	**98.55**	98.51	98.58	**98.55**
Style	91.65	91.70	91.67	98.35	98.45	98.40	98.41	98.50	**98.45**
Material	90.47	90.91	90.69	98.89	99.02	**98.95**	98.64	98.84	98.74
Size	80.32	79.44	79.88	96.95	97.60	97.28	97.32	97.50	**97.41**
Capacity	86.94	86.39	86.67	98.09	98.35	**98.22**	98.19	98.19	98.19
Black Tea Variety	100.00	80.00	88.89	100.00	100.00	**100.00**	100.00	100.00	**100.00**
Staple Type	90.91	90.91	90.91	100.00	100.00	**100.00**	100.00	100.00	**100.00**
Web Pattern	94.74	94.74	94.74	100.00	100.00	**100.00**	100.00	100.00	**100.00**
Cabinet Configuration	94.74	75.00	83.72	100.00	91.67	**95.65**	100.00	91.67	**95.65**
Power Consumption	43.75	43.75	43.75	100.00	100.00	**100.00**	100.00	100.00	**100.00**
All Attributes	86.42	74.00	79.73	98.06	98.21	98.14	98.27	98.37	**98.32**

Table 5: Individual results on each selected attribute and average results on all attributes. OpenTag is used for each selected attribute and ADOpenTag is used for all attributes.

- **Few-shot Learning** To study the impact of different number of training examples, we explore the model behavior with only a few training examples, namely few-shot learning.

We randomly split the dataset into train:eval:test = 8:1:1[3]. Note that OpenTag trains a single model per attribute on the Selected Attributes. We use its extended version, **ADOpenTag**, to train another model on all attributes. The number of examples in each set is reported in Table 4.

5.3 Metrics

Following previous works [54, 56, 59], we use Precision, Recall, and F1 score as evaluation metrics. Specifically, for each attribute annotation, when the ground truth is No attribute value, the model can predict No value (**NN**) or some incorrect Value (**NV**); when the ground truth has attribute Values, the model can predict No value (**VN**), Correct values (**VC**) or Wrong values (**VW**). We define **NN**, **NV**, **VN**, **VC**, and **VW** as the number of examples for the above five cases. The precision and recall can be computed as:

$$\mathbf{P} = \frac{\mathbf{VC}}{\mathbf{NV} + \mathbf{VC} + \mathbf{VW}}, \quad \mathbf{R} = \frac{\mathbf{VC}}{\mathbf{VN} + \mathbf{VC} + \mathbf{VW}}.$$

The **F1** score is calculated as $\mathbf{2PR}/(\mathbf{P} + \mathbf{R})$.

5.4 Results on Selected Attributes

To study and compare the baseline model performances on individual attributes, we select five head attributes and five tail attributes. Note that an attribute can be shared across multiple categories. It can be seen from Figure 3 that the number of categories containing the attribute and the number of the examples having the attribute are positively correlated. As aforementioned, we randomly select five head attributes, including *Type*, *Style*, *Material*, *Size*, and *Capacity*. It is clear that these attributes have more general meaning which are commonly shared across categories. The tail attributes have a small number of examples and often only appear in 1-2 categories. In order to have a meaningful evaluation on tail attributes, we also constrain the number of evaluation examples to be larger than 15. The tail attributes include *Black Tea Variety*, *Staple Type*, *Web Pattern*, *Cabinet Configuration* and *Power Consumption*.

[3]We randomly sample up to 640k examples for training for faster convergence.

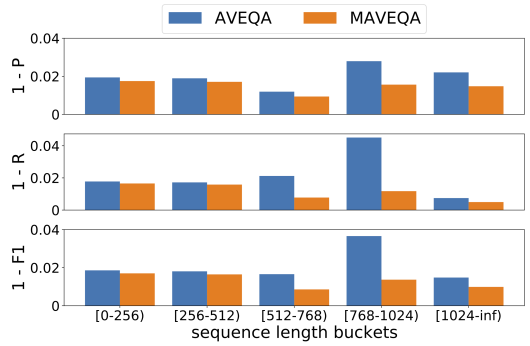

Figure 5: Precision, Recall, F1 errors within each bucket of example sequence length for all attributes.

The results of the OpenTag, AVEQA, and MAVEQA models on these selected attributes are reported in Table 5. From these comparison results, we can see that the AVEQA and MAVEQA models perform consistently better than OpenTag on all head and tail attributes. The reason is that both AVEQA and MAVEQA use the advanced Transformer architecture which have been shown to outperform BiLSTM-Attention-CRF architecture. Moreover, the knowledge can be transferred among different attributes in AVEQA and MAVEQA, since they jointly encode the attribute and the product across all attributes. It also can be seen that AVEQA and MAVEQA achieve similar results on these selected attributes. We found out, by examining the evaluation examples, that the product sequences of these selected attributes are not long, and thus AVEQA and MAVEQA are equally effective when modeling short sequences. We will discuss more on the effect of product sequence length in the next section. Another interesting observation is that the results of MAVEQA on the tail attributes are even better (F1 scores are almost 100%) than those on the head attributes. This is because although tail attributes have relatively small training examples, their attribute-value patterns are much simpler than those in the head attribute. For example, *Black Tea Variety* is a specific attribute in category *Tea & Infusions*, which only has three possible values, i.e. *Ceylon*, *Darjeeling* and *Earl Grey*. On the other hand, the attribute *Type* appears in more than 40 categories with more than 500

Zero-shot Attributes	AVEQA			MAVEQA		
	P (%)	R (%)	F (%)	P (%)	R (%)	F (%)
Special Occasion	62.13	50.16	**55.51**	10.06	0.80	1.48
Resolution	80.95	38.55	52.23	85.56	44.94	**58.93**
Compatibility	0.00	0.00	0.00	0.00	0.00	0.00
Number of Sinks	39.10	29.80	33.82	48.57	38.97	**43.24**
Food Processor Capacity	81.97	92.34	86.85	83.50	98.85	**90.53**

Table 6: Results on zero-shot attributes.

different values. However, MAVEQA is still able to achieve very high performance on the head attributes.

5.5 Results on All Attributes

The average results over all attributes are reported in Table 5. We observe similar result patterns to those in the selected attributes. It can be seen that MAVEQA performs slightly better than AVEQA. Our hypothesis is that for those examples with multiple product sources and long sequences, MAVEQA is more effective in capturing the attribute-value relations via modeling the complete structure and long sequence. In contrast, AVEQA simply concatenates and trims the text from different sources and treats it as a single sequence. To further investigate along this direction, we divide the evaluation examples into different buckets w.r.t. the sequence length of the example, and compute the metrics in each bucket for AVEQA and MAVEQA. The precision, recall and F1 gap/error results are shown in Figure 5. It can be seen that with example sequence length exceeding max sequence length of the models, both precision and recall errors for AVEQA go up, while the errors remain the same for MAVEQA. This observation validates that MAVEQA is indeed more effective on examples with longer sequences.

5.6 Results on Zero-shot Attributes

We conduct zero-shot extraction experiments to evaluate the generalization ability of AVEQA and MAVEQA on unseen attributes. The zero-shot extraction results are reported in Table 6. There are several interesting observations from this table. First, it is clear that the overall performance of both methods on these zero-shot attributes are much worse compared with those results in Table 5, which is consistent with our expectation as there are no training examples for the zero-shot attributes. Second, AVEQA achieves better results than MAVEQA on *Special Occasion*. Our hypothesis is that AVEQA and MAVEQA are trained with different pre-trained models, i.e., AVEQA is initialized with the pre-trained BERT [9] while MAVEQA is initialized from ETC [3]. Previous works [14, 27, 54] have shown that the pre-trained model is crucial for the fine-tuned model to perform well on zero-shot learning. However, the pre-trained ETC model removes all examples with less than 10 sentences since it focuses on long sequences, which might have eliminated a certain amount of knowledge on attribute *Special Occasion*, and thus does not perform well. Finally, AVEQA and MAVEQA achieve 0 scores in terms of all metrics on *Compatibility*, which indicates that it is a truly difficult zero-shot attribute. Both pre-trained models and the training examples of other attributes are lacking information about *Compatibility*, resulting in 0 extractions. We believe there is deep knowledge and information that are uncovered in zero-shot extraction. The MAVE dataset can serve as a rich resource to drive research in this direction.

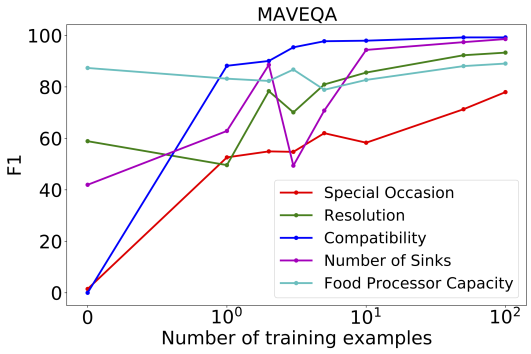

Figure 6: Results on Few-shot learning.

5.7 Results on Few-shot Learning

To study the effectiveness of MAVEQA with few training examples, we conduct few-shot experiments by varying the number of training examples, k, from {1, 2, 3, 5, 10, 50, 100} on all zero-shot attributes. For each run, we first randomly separate out 100 examples per attribute, then we randomly select k examples from the 100 examples. We perform fine-tuning for 20K steps over these selected examples initializing from the model trained on all attributes (exclude zero-shot attributes). The F1 scores of MAVEQA on few-shot experiments are shown in Figure 6. It is not surprising to see that the F1 scores increase with more training examples. We can also observe that the F1 scores on most attributes are able to reach a reasonably high value, above 90%, when the number of training examples increases to 100. We notice that the F1 score remains flattened for attribute *Food Processor Capacity*. By analyzing the errors, we found that there are a certain amount of false negatives in the evaluation set, where the extractions are actually correct but the attribute value annotations are missing. Improving the attribute coverage on MAVE is definitely one of the future directions.

6 CONCLUSION

In this paper, we introduce the Multi-source Attribute Value Extraction (MAVE) dataset – the largest, multi-source, diverse, context-rich, clean dataset. By extracting and verifying attribute values from over 2.2 million multi-source product profiles, MAVE provides a large set of 3 million attribute-value annotations. We establish benchmarks on MAVE with several state-of-the-art methods, including a novel approach proposed in this work that models the multi-source and long product profiles. We also empirically demonstrated the use of this dataset on a very challenging task - zero-shot attribute value extraction, which is not fully exploited by existing datasets. We believe this can serve as a rich resource to drive research in the attribute value extraction domain for years to come and enable the community to build better and more generalized models. MAVE can potentially be used as a pretraining dataset in various downstream tasks, including product embedding learning, attribute grounded questions answering, product search and recommendation, etc.

REFERENCES

[1] A. Ahuja, N. Rao, S. Katariya, K. Subbian, and C. K. Reddy. Language-agnostic representation learning for product search on e-commerce platforms. In *WSDM*, pages 7–15, 2020.

[2] Q. Ai, Y. Zhang, K. Bi, X. Chen, and W. B. Croft. Learning a hierarchical embedding model for personalized product search. In *SIGIR*, pages 645–654, 2017.

[3] J. Ainslie, S. Ontañón, C. Alberti, V. Cvicek, Z. Fisher, P. Pham, A. Ravula, S. Sanghai, Q. Wang, and L. Yang. ETC: encoding long and structured inputs in transformers. In *EMNLP*, pages 268–284, 2020.

[4] D. Carmel, L. Lewin-Eytan, and Y. Maarek. Product question answering using customer generated content - research challenges. In *SIGIR*, pages 1349–1350, 2018.

[5] K. Chen, L. Feng, Q. Chen, G. Chen, and L. Shou. EXACT: attributed entity extraction by annotating texts. In *SIGIR*, pages 1349–1352, 2019.

[6] L. Chiticariu, R. Krishnamurthy, Y. Li, F. Reiss, and S. Vaithyanathan. Domain adaptation of rule-based annotators for named-entity recognition tasks. In *EMNLP*, pages 1002–1012, 2010.

[7] J. P. C. Chiu and E. Nichols. Named entity recognition with bidirectional lstm-cnns. *TACL*, 4:357–370, 2016.

[8] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. P. Kuksa. Natural language processing (almost) from scratch. *J. Mach. Learn. Res.*, 12:2493–2537, 2011.

[9] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, pages 4171–4186, 2019.

[10] A.-H. Dirie. *Extracting Diverse Attribute-Value Information from Product Catalog Text via Transfer Learning*. PhD thesis, 2017.

[11] S. Gao, Z. Ren, Y. E. Zhao, D. Zhao, D. Yin, and R. Yan. Product-aware answer generation in e-commerce question-answering. In *WSDM*, pages 429–437, 2019.

[12] R. Ghani, K. Probst, Y. Liu, M. Krema, and A. E. Fano. Text mining for product attribute extraction. *SIGKDD Explorations*, 8(1):41–48, 2006.

[13] V. Gopalakrishnan, S. P. Iyengar, A. Madaan, R. Rastogi, and S. H. Sengamedu. Matching product titles using web-based enrichment. In *CIKM*, pages 605–614, 2012.

[14] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[15] Z. Huang, W. Xu, and K. Yu. Bidirectional LSTM-CRF models for sequence tagging. *CoRR*, abs/1508.01991, 2015.

[16] K. S. D. Ishwari, A. K. R. R. Aneeze, S. Sudheesan, H. J. D. A. Karunaratne, A. Nugaliyadde, and Y. Mallawarachchi. Advances in natural language question answering: A review. *CoRR*, abs/1904.05276, 2019.

[17] R. L. L. IV, S. Humeau, and S. Singh. Multimodal attribute extraction. In *6th Workshop on Automated Knowledge Base Construction, AKBC@NIPS*, 2017.

[18] M. Jain, S. Bhattacharya, H. Jain, K. Shaik, and M. Chelliah. Learning cross-task attribute - attribute similarity for multi-task attribute-value extraction. In *Proceedings of The 4th Workshop on e-Commerce and NLP*, 2021.

[19] R. Kalyani, G. Pawan, and P. Manish. Attribute value generation from product title using language models. In *Proceedings of The 4th Workshop on e-Commerce and NLP*, 2021.

[20] G. Karamanolakis, J. Ma, and X. L. Dong. Txtract: Taxonomy-aware knowledge extraction for thousands of product categories. In *ACL*, pages 8489–8502, 2020.

[21] L. Kodra and E. K. Meçe. Question answering systems: A review on present developments, challenges and trends. *International Journal of Advanced Computer Science and Applications*, 8(10.14569), 2017.

[22] Z. Kozareva, Q. Li, K. Zhai, and W. Guo. Recognizing salient entities in shopping queries. In *ACL*, pages 107–111, 2016.

[23] A. Kulkarni, K. Mehta, S. Garg, V. Bansal, N. Rasiwasia, and S. H. Sengamedu. Productqna: Answering user questions on e-commerce product pages. In *WWW*, pages 354–360, 2019.

[24] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer. Neural architectures for named entity recognition. In *NAACL-HLT*, pages 260–270, 2016.

[25] T. Le and H. W. Lauw. Explainable recommendation with comparative constraints on product aspects. In *WSDM*, pages 967–975, 2021.

[26] O. Levy, M. Seo, E. Choi, and L. Zettlemoyer. Zero-shot relation extraction via reading comprehension. In *CoNLL*, pages 333–342, 2017.

[27] Z. Li and D. Hoiem. Learning without forgetting. *TPAMI*, 40(12):2935–2947, 2018.

[28] R. Lin, X. He, J. Feng, N. Zalmout, Y. Liang, L. Xiong, and X. L. Dong. PAM: understanding product images in cross product category attribute extraction. In *SIGKDD*, pages 3262–3270, 2021.

[29] X. Ling and D. S. Weld. Fine-grained entity recognition. In *AAAI*, pages 94–100, 2012.

[30] D. Liu, Y. Cui, L. Yan, C. Mousas, B. Yang, and Y. V. Chen. Densernet: Weakly supervised visual localization using multi-scale feature aggregation. In *AAAI*, pages 6101–6109, 2021.

[31] Y. Liu, S. Zhang, R. Song, S. Feng, and Y. Xiao. Knowledge-guided open attribute value extraction with reinforcement learning. In *EMNLP*, pages 8595–8604, 2020.

[32] X. Ma and E. H. Hovy. End-to-end sequence labeling via bi-directional lstm-cnns-crf. In *ACL*, pages 1064–1074, 2016.

[33] P. Martinez-Gomez, G. Papachristoudis, J. Blauvelt, E. Rachlin, and S. Simhon. Enhancement and analysis of tars few-shot learning model for product attribute extraction from unstructured text. In *KDD Workshop on Pretraining: Algorithms, Architectures, & Applications*, 2021.

[34] Z. Meng, S. Liang, J. Fang, and T. Xiao. Semi-supervisedly co-embedding attributed networks. In *NeurIPS*, pages 6504–6513, 2019.

[35] A. Mikheev, M. Moens, and C. Grover. Named entity recognition without gazetteers. In *EACL*, pages 1–8, 1999.

[36] A. More. Attribute extraction from product titles in ecommerce. *CoRR*, abs/1608.04670, 2016.

[37] D. Nadeau and S. Sekine. A survey of named entity recognition and classification. *Lingvisticæ Investigationes*, 30(1):3–26, 2007.

[38] J. Ni, J. Li, and J. J. McAuley. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *EMNLP-IJCNLP*, pages 188–197, 2019.

[39] G. Pan, Y. Yao, H. Tong, F. Xu, and J. Lu. Unsupervised attributed network embedding via cross fusion. In *WSDM*, pages 797–805, 2021.

[40] N. Peng, H. Poon, C. Quirk, K. Toutanova, and W. Yih. Cross-sentence n-ary relation extraction with graph lstms. *TACL*, 5:101–115, 2017.

[41] P. Petrovski and C. Bizer. Extracting attribute-value pairs from product specifications on the web. In *ICWI*, pages 558–565, 2017.

[42] P. Petrovski, V. Bryl, and C. Bizer. Learning regular expressions for the extraction of product attributes from e-commerce microdata. In *LD4IE*, pages 43–54, 2014.

[43] K. Probst, R. Ghani, M. Krema, A. E. Fano, and Y. Liu. Semi-supervised learning of attribute-value pairs from product descriptions. In *IJCAI*, pages 2838–2843, 2007.

[44] D. Putthividhya and J. Hu. Bootstrapped named entity recognition for product attribute extraction. In *EMNLP*, pages 1557–1567, 2011.

[45] D. Qiu, L. Barbosa, X. L. Dong, Y. Shen, and D. Srivastava. DEXTER: large-scale discovery and extraction of product specifications on the web. *PVLDB*, 8(13):2194–2205, 2015.

[46] M. Rezk, L. A. Alemany, L. Nio, and T. Zhang. Accurate product attribute extraction on the field. In *ICDE*, pages 1862–1873, 2019.

[47] S. Riedel, L. Yao, and A. McCallum. Modeling relations and their mentions without labeled text. In *ECML/PKDD*, pages 148–163, 2010.

[48] K. Shinzato and S. Sekine. Unsupervised extraction of attributes and their values from product description. In *IJCNLP*, pages 1339–1347, 2013.

[49] F. M. Suchanek, G. Ifrim, and G. Weikum. Combining linguistic and statistical analysis to extract relations from web documents. In *SIGKDD*, 2006.

[50] C. Sun, H. Liu, M. Liu, Z. Ren, T. Gan, and L. Nie. LARA: attribute-to-feature adversarial learning for new-item recommendation. In *WSDM*, pages 582–590, 2020.

[51] D. Vandic, J. van Dam, and F. Frasincar. Faceted product search powered by the semantic web. *Decision Support Systems*, 53(3):425–437, 2012.

[52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NIPS*, pages 5998–6008, 2017.

[53] Q. Wang, B. Kanagal, V. Garg, and D. Sivakumar. Constructing a comprehensive events database from the web. In *CIKM*, pages 229–238, 2019.

[54] Q. Wang, L. Yang, B. Kanagal, S. Sanghai, D. Sivakumar, B. Shu, Z. Yu, and J. Elsas. Learning to extract attribute value from product via question answering: A multi-task approach. In *SIGKDD*, pages 47–55, 2020.

[55] Y. W. Wong, D. Widdows, T. Lokovic, and K. Nigam. Scalable attribute-value extraction from semi-structured text. In *ICDM Workshops*, pages 302–307, 2009.

[56] H. Xu, W. Wang, X. Mao, X. Jiang, and M. Lan. Scaling up open tagging from tens to thousands: Comprehension empowered attribute value extraction from product title. In *ACL*, pages 5214–5223, 2019.

[57] S. Xu, H. Li, P. Yuan, Y. Wang, Y. Wu, X. He, Y. Liu, and B. Zhou. K-PLUG: knowledge-injected pre-trained language model for natural language understanding and generation in e-commerce. *CoRR*, abs/2104.06960, 2021.

[58] H. Yan, T. Gui, J. Dai, Q. Guo, Z. Zhang, and X. Qiu. A unified generative framework for various NER subtasks. In *ACL/IJCNLP*, pages 5808–5822, 2021.

[59] J. Yan, N. Zalmout, Y. Liang, C. Grant, X. Ren, and X. L. Dong. Adatag: Multi-attribute value extraction from product profiles with adaptive decoding. In *ACL/IJCNLP*, pages 4694–4705, 2021.

[60] D. Zeng, K. Liu, S. Lai, G. Zhou, and J. Zhao. Relation classification via convolutional deep neural network. In *COLING*, pages 2335–2344, 2014.

[61] J. Zhao, Z. Guan, and H. Sun. Riker: Mining rich keyword representations for interpretable product question answering. In *SIGKDD*, pages 1389–1398, 2019.

[62] G. Zheng, S. Mukherjee, X. L. Dong, and F. Li. Opentag: Open attribute value extraction from product profiles. In *SIGKDD*, pages 1049–1058, 2018.

[63] T. Zhu, Y. Wang, H. Li, Y. Wu, X. He, and B. Zhou. Multimodal joint attribute prediction and value extraction for e-commerce product. In *EMNLP*, pages 2129–2139, 2020.

A APPENDIX

A.1 Implementation Details

For all these baselines, we use the same output layer which is a single dense layer with sigmoid activation to predict token scores, and we use the same English uncased WordPiece vocabulary as in BERT. For **ADOpenTag** and **AVEQA**, we use the input format of "[CLS] category tokens [SEP] attribute tokens [SEP] all source tokens [SEP]". For **OpenTag** and **MAVEQA**, we do not use any special tokens such as [CLS], [SEP]. This is to balance between minimizing the difference of the input formats and better utilizing pretrained models. Table 7 contains the hyper-parameters details for training the **(AD)OpenTag**, **AVEQA**, and **MAVEQA** models.

Hyper-parameters	Value
Common	
batch size	128
iterations	200,000
optimizer	Adam
Adam β_1, β_1, ϵ	0.9, 0.99, $1e^{-6}$
learning rate schedule	linear decay
initial learning rate	$3e^{-5}$
end learning rate	0.0
learning rate warmup steps	10,000
vocab size	30522
(AD)OpenTag	
max sequence length	512
word embedding size	768
unidirectional LSTM units	512
LSTM inputs dropout	0.0
LSTM recurrent dropout	0.4
attention dropout	0.0
word embedding layer init	TFHub BERT-base
AVEQA	
max sequence length	512
transformer layers	12
transformer attention heads	12
transformer hidden dimension	768
pretrained checkpoint	TFHub BERT-base
MAVEQA	
max long sequence length	1024
max global sequence length	64
transformer layers	12
transformer attention heads	12
transformer hidden dimension	768
local attention radius	84
pretrained checkpoint	etc_base_2x_pretrain

Table 7: Model Hyper-parameters details.

Models	# params	Hardware	Training time
OpenTag	28.7M	32 core TPU v3	7.1h
AVEQA	109M	32 core TPU v3	2.7h
MAVEQA	166M	64 core TPU v3	7.2h

Table 8: Model parameters and training hardware.

A.2 Number of Model Parameters, Hardware and Training Cost

The number of model parameters, hardwares to train the model and training time are shown in Table 8. The larger number of parameters for **MAVEQA** is due to the parameters for global nodes in the ETC model. Since **MAVEQA** uses a longer sequence length and has more parameters, we use more TPU cores to train. We use Tensorflow distribute strategy with SUM loss reduction. We note that this is not an issue since we are using Adam optimizer which normalizes the gradients by the second raw momentum estimate.

Note that even **(AD)OpenTag** has much smaller number of parameters than **AVEQA**, it takes much longer time to train on TPUs, which may due to the recurrent nature of its LSTM component. **MAVEQA** takes much longer than **AVEQA**, which may due to a longer sequence length and the implementation of ETC's local attention on TPU being not as efficient as full attention.

A.3 Additional Results on Few-Shot Learning

We provide more results on the few-shoot learning using **AVEQA** in Figure 7, which has a similar trend as **MAVEQA**.

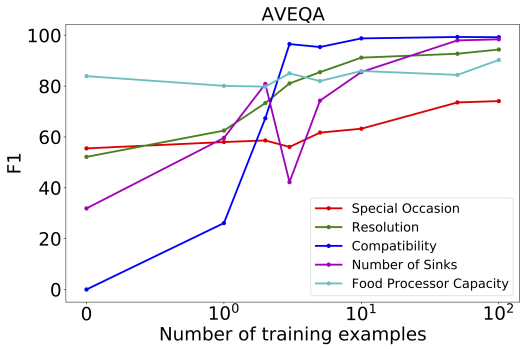

Figure 7: Impact of different numbers of training examples for the AVEQA model.

A.4 Ablation Study

We also used different sizes of BERT backbones for **AVEQA**. The BERT parameters are initialized from TFHub pretrained checkpoints. The results on random split of all attributes are shown in Table 9. We can see that as model size increases, the results constantly become better. However, the BERT-large results are just close to the **MAVEQA** results in Table 5 in the main text, which suggests the advantage of **MAVEQA** over **AVEQA**.

BERT sizes	P (%)	R (%)	F (%)
BERT-2L	91.60	90.14	90.87
BERT-4L	96.78	96.92	96.85
BERT-6L	97.98	98.17	98.07
BERT-8L	98.10	98.27	98.18
BERT-base	98.06	98.21	98.14
BERT-large	**98.29**	**98.33**	**98.31**

Table 9: Ablation study of different sizes of BERT backbone in AVEQA. The metrics are for random split on all attributes.